



**NATIONAL INSTITUTE OF BUSINESS
MANAGEMENT**

School of Computing & Engineering

**BSc (Hons) Ethical Hacking and Network Security
Higher National Diploma in Network Engineering (HNDNE)
Batch – 252F**

**NetWatch: A Network Monitoring Toolkit for
Serendib Logistics (Pvt) Ltd**

Module: Network Programming Design

Lecturer: Malith Wijethunga

Group members:

COHNDNE252F-004 K.A Baylon

COHNDNE252F-005 S.M.J Nawarathna

COHNDNE252F-007 J A K C Jayasinghe

COHNDNE252F-014 A D T Silva

TABLE OF CONTENTS

TABLE OF FIGURES	2
1. INTRODUCTION & PROBLEM DEFINITION	4
2. REQUIREMENTS ANALYSIS	5
2.1 Stakeholders	5
2.2 Functional requirements	5
2.3 Non – functional Requirements	6
2.4 Constraints and Assumptions	7
3. DESIGN	8
3.1 Network architecture	8
3.2 Program Architecture	9
3.3 Data contract	9
3.4 Exit-code scheme	10
4. SHELL IMPLEMENTATION	10
5. PYTHON IMPLEMENTATION	14
6. INTEGRATION	20
6.1 Integration and pipeline	20
6.2 Technology Justification	22
7. TESTING & RESULTS	23
7.1 Test Plan	23
7.2 Results table	26
7.3 Performance testing	28
8. ERROR HANDLING	30
8.1 Error-Handling Condition Matrix	30
8.2 Error Handling Discussion	30
9. SECURITY CONSIDERATIONS	32
10. CRITICAL EVALUATION & CONCLUSION	33
REFERENCES	35
APPENDIX A – Code Listing	36
APPENDIX B	37
APPENDIX C – AI Use Declaration	40

TABLE OF FIGURES

Figure 1: Network Architecture	8
Figure 2: Program Architecture	9
Figure 3: FR-S01:Inventory validation netwatch.sh lines 110	10
Figure 4: header is read and checked	11
Figure 5: Each row is split into five expected fields.....	11
Figure 6: basic IPv4 validation	11
Figure 7: Invalid port validation	11
Figure 8: commandline agruments.....	12
Figure 9: timeout validation set to a positive integer.....	12
Figure 10: output directory created and checked for write permissions	12
Figure 11: ping_host() - helper.sh.....	12
Figure 12: Returned status and RTT	12
Figure 13: netwatch.sh the strict mode is enabled , use a script relative path to source the shared library	13
Figure 14: log_error() in helper.sh.....	13
Figure 15: Geting arguments manually parsed from the list.....	14
Figure 16: exit code - netwatch.py.....	14
Figure 17: valid modes - netwatch.py	14
Figure 18: error for Invalid arguments & usage message.....	14
Figure 19: csv.reader to read the common inventory - netwatch.py.....	15
Figure 20: validates the ip headers , ports and ipv4 address.....	15
Figure 21: dataclasses	15
Figure 22: TCP socket and set timeouts and returns error value	16
Figure 23: Contains the information needed bythe reports -netwatch.py	16
Figure 24: sequential and concurrent scan modes	17
Figure 25: Results get sorted.....	17
Figure 26: compare the current and previous services.....	17
Figure 27: data contract used	18
Figure 28: count states	18
Figure 29: Exit code.....	18
Figure 30: summary end including the status	18
Figure 31: status_server.py status service.....	18

Figure 32: client socket - status_client.py.....	19
Figure 33: pipeline flow.....	21

1. INTRODUCTION & PROBLEM DEFINITION

Serendib Logistics (Pvt) Ltd is a company in colombo that operates freight forwarding and warehousing from a single server room. Their internal network is small but its crucial for operational acivities. It consists of a gateway, an intranet web server, a warehouse management application server, a shared file server and some workstations. The company employs one IT officer Dilani, whose responsible for monitoring the health of this entire system manually by pinging each server from a spreadsheet list every morning and separately checking whether the intranet loads in a browser.

This manual process is already inefficient and has caused a costly failure. The warehouse management application has stopped responding on its service port at approximately 19:40 on a recent evening because the routine check only focused on reachability at the network level hence it could not be distinguished the availability of the specific service. Due to this up until the morning nobody noticed that the application had stopped, by the time three lorries had been loaded against paperwork that the system had not actually been able to update. The core problem is not that the company lacks monitoring effort but that the monitoring is not very specific and has no procedures beyond a single morning check.

NetWatch is our solution to this challenge. It consists of two sections that work together a Bash component (Part A) responsible for host level reachability and historical reporting and a Python component (Part B) account for service level socket checks, concurrent scanning, change detection and network accessible status publishing designed to run without requiring Dilani. Netwatch is able to run on Dilani's existing Linux workstation without additional requirements. The success of this project is determined by its ability to verify the reachability of each host at the moment, the availability of the specific services and has any changes occur since the last check. It also means that this tool must be able to keep performing its functions even if a host goes offline, an unattended overnight run or a malformed row typed by a non programmer without silently stopping or failing.

2. REQUIREMENTS ANALYSIS

2.1 Stakeholders

Through the scenario given Dilani is a primary stakeholder but she is not the only person NetWatch affects. The following table depicts some of the stakeholders of NetWatch;

Stakeholder	Interest in the system	What they need
Dilani (IT Officer)	Manually operates this work and was directly affected by the undetected outage	A tool she can run unattended , read the output without a terminal session , operates even if a device is edited or offline
Warehouse staff	Depends on the warehouse management application being available	Early accurate detection of service level failure not just host level.
Management board	Bears financial losses due to undetected failures	Historical evidence of availability
Non technical staff	Will add , remove or mistype inventory entries	A tool that rejects bad rows with a clear reason rather than failing or ignoring the mistakes.
Future developers	May need to add future developments or scale the project	Readable , clear code and documentations.

2.2 Functional requirements

- The functional requirements are those specified within the coursework brief, FR-S01 – FR-S05 for the bash component , FR-P01 – FR-P08 for the python component and INT-01 – INT-03 governing integration between them. The below points summaries the functional requirements implemented in the system;
- FR-S01–FR-S05 (Bash): inventory validation, CLI parsing, reachability probing, history/log writing and report generation are each testable by inspecting the corresponding exit code after a run against a known good and a known bad inventory.

- FR-P01–FR-P08 (Python): CLI validation, inventory loading, single-service socket checks, sequential and concurrent scanning, persistence and change detection, exit-code reporting, status publishing and (Group 1 only) socket-based status retrieval are each testable independently, since `netwatch.py`, `status_server.py` and `status_client.py` are separate guarded entry points.
- INT-01–INT-03: shared inventory validation, orchestrated invocation with exit code handling, and the merged report are testable by running the full pipeline end to end and checking that a row rejected by one component is rejected in the same way by the other, and that Part B's results appear inside Part A's consolidated report.

2.3 Non – functional Requirements

The following table contains the main non functional requirements of our tool NetWatch ;

Category	Requirement
Performance	Concurrent service checks must be complete comparatively faster than sequential checks.
Reliability	A single unreachable host or closed port must never stop a run
Usability	Dilani must be able to see current status without needing to open a terminal.
Maintainability	The device inventory must be editable by a non programmer without breaking either component
Security	The tool must not expose the internal network topology beyond the workstation it runs on.

2.4 Constraints and Assumptions

Constraints

- Only 127.0.0.1/localhost and any IP range issued in writing by the lecturer may be targeted no other address may appear in the submitted inventory.
- No packages beyond the Python standard library and Bash / coreutils may be installed; both components must run on a clean machine within twenty minutes using only what is documented in README.md.
- Both components must accept config/inventory.csv without modification to its column meanings and must apply the same validation rules to it (INT-01).

Assumptions

- The admin workstation's clock is correct and consistent between runs.
- Dilani's workstation has Python 3 and Bash available out of the box, as stated in the brief.
- A single, non-clustered admin workstation is sufficient and no requirement for the monitoring tool itself to be highly available.

3. DESIGN

3.1 Network architecture

The network architecture diagram depicts the admin workstation having both the NetWatch components together the host machines having their corresponding simulated IP address and port numbers (config/inventory.csv) and the status service with all connections initiated outwardly from the admin workstation only a web browser or status_client.py can initiate a connection to 127.0.0.1:8888.

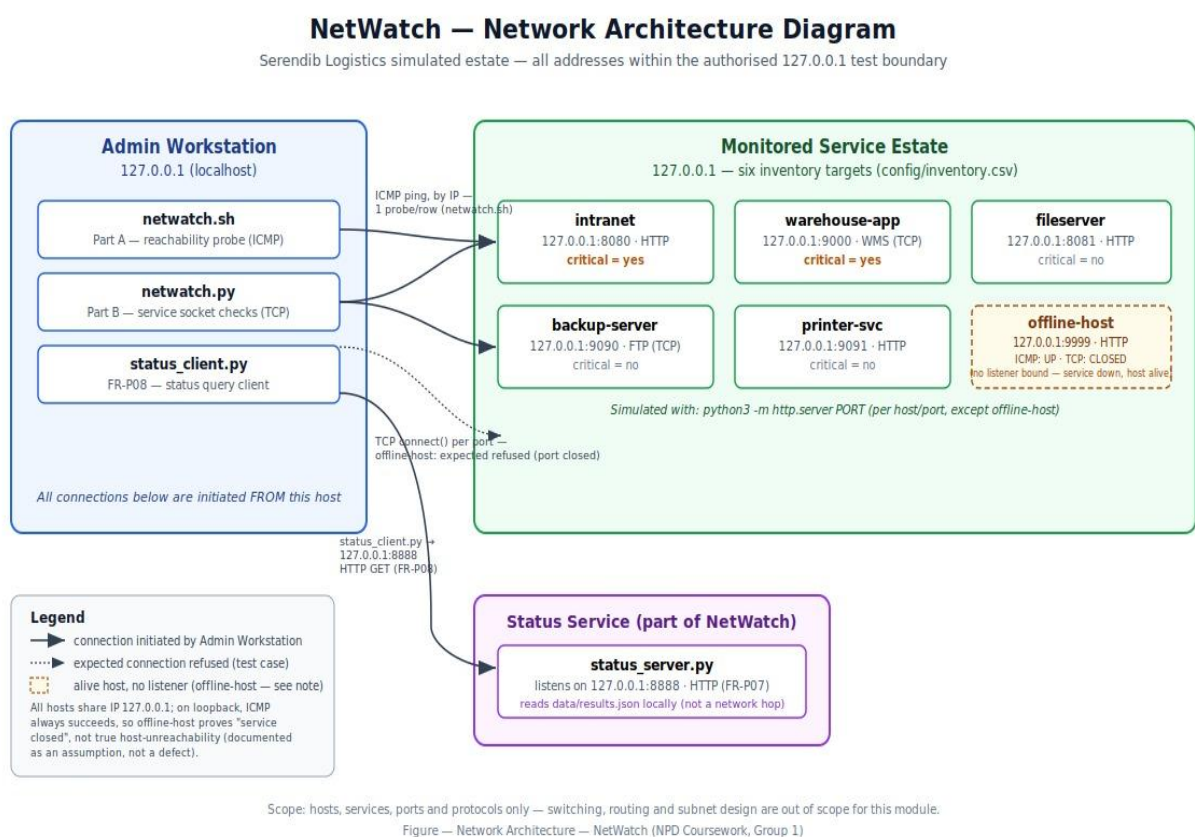


Figure 1: Network Architecture

3.2 Program Architecture

The following diagram depicts the two components of Netwatch part A (bash) and part B (python), the files each reads and writes and the three integration points (INT-01-INT-03) where they exchange data.

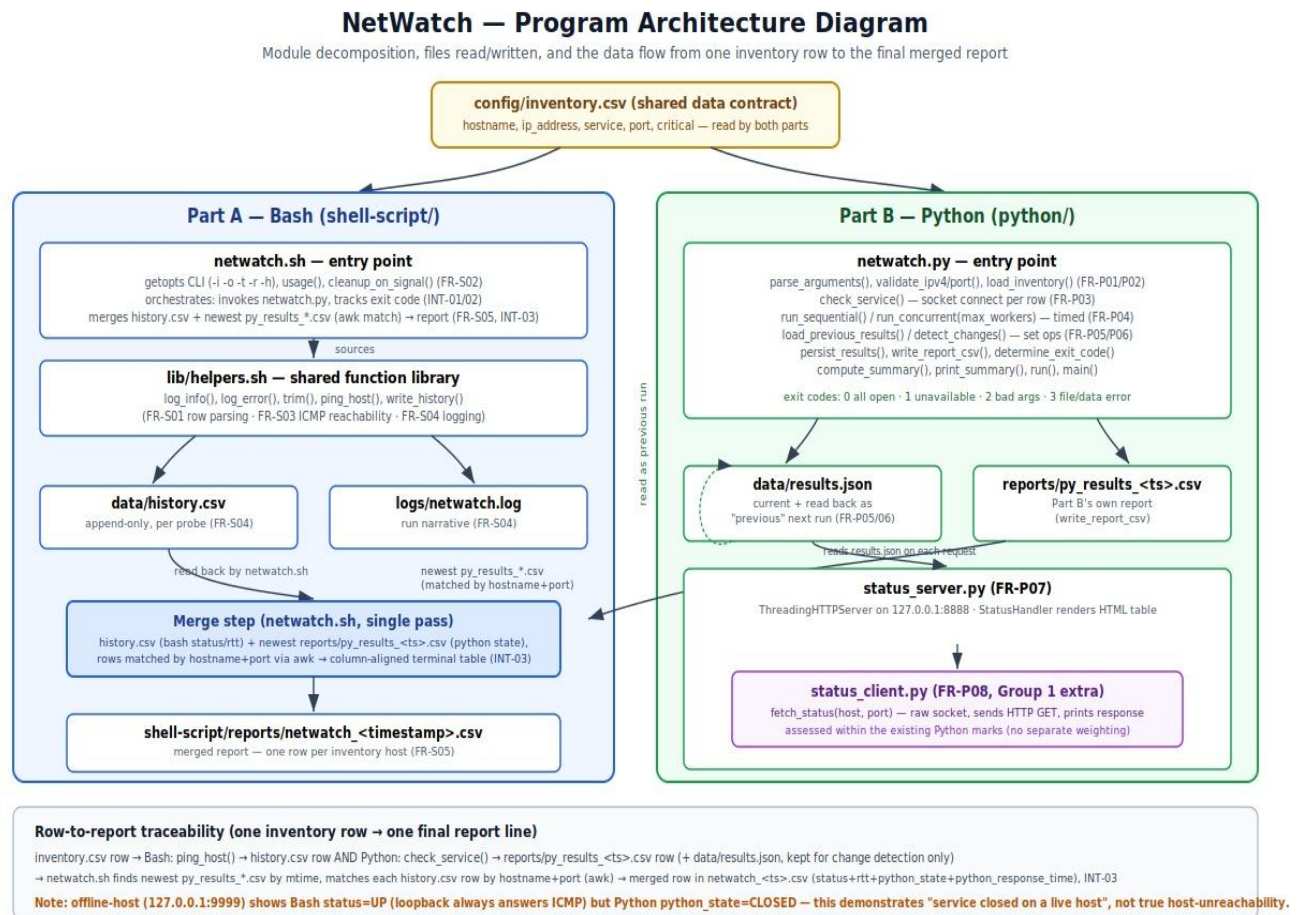


Figure — Program Architecture — NetWatch (NPD Coursework, Group 1)

Figure 2: Program Architecture

3.3 Data contract

The table below depicts the agreed format shared between Part A and Part. It tells the producer what fields to write and consumer how to read them.

Position	Field	Example
1	timestamp	2026-09-27T19:26:21+05:30
2	hostname	intranet
3	ip address	127.0.0.1
4	service	http
5	port	8080
6	critical	yes
7	state	OPEN
8	reponse_time	0.0011
9	Error_detail	Connection refused

3.4 Exit-code scheme

NetWatch's exit code scheme is defined in netwatch.py using the following constants;

- EXIT_SUCCESS = 0 → All checks are completed successfully
- EXIT_WARNING = 1 → At least one service is either closed , timed out or errored
- EXIT_USAGE_ERROR = 2 → Invaild command line argument
- EXIT_FATAL_ERROR = 3 → Fatal file or data error
- EXIT_INTERRUPTED = 130 → Interrupted with ctr + c

4. SHELL IMPLEMENTATION

FR-S01: Inventory validation

The script reads the inventory file that specifies -i or uses the default inventory.csv. It checks whether the file exists and is readable before processing it.

```

110 if [[ ! -r "${INVENTORY_PATH}" ]]; then
111     echo "Error: inventory '${INVENTORY_PATH}' is missing or unreadable" >&2
112     exit 3
113 fi

```

Figure 3: FR-S01:Inventory validation netwatch.sh lines 110

The header is read and verified whether each row is able to be split into expected fields.

```
123 IFS= read -r header < "${INVENTORY_PATH}" || true
124 header="$(trim "${header}")"
125 if [[ "${header}" != "hostname,ip_address,service,port,critical" ]]; then
126     echo "Error: inventory header is invalid; expected hostname,ip_address,service,port,critical" >&2
127     exit 3
128 fi
```

Figure 4: header is read and checked

```
138 IFS=',' read -r hostname address service port critical extra <<< "${line}"
139 hostname="$(trim "${hostname}")"
140 address="$(trim "${address}")"
141 service="$(trim "${service}")"
142 port="$(trim "${port}")"
143 critical="$(trim "${critical}")"
```

Figure 5: Each row is split into five expected fields

A basic ipv4 format is validated and each individual octets are then checked to ensure that they don't exceed 255.

```
150 if ! [[ "${address}" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]]; then
151     log_error "Rejected inventory line ${line_number}: invalid IPv4 address '${address}'"
152     rejected=$((rejected + 1))
153     continue
154 fi
155 valid_address=1
156 IFS='.' read -r octet1 octet2 octet3 octet4 <<< "${address}"
157 for octet in "${octet1}" "${octet2}" "${octet3}" "${octet4}"; do
158     if ((10#${octet} > 255)); then valid_address=0; fi
159 done
```

Figure 6: basic IPv4 validation

The port is checked as a number between 1 – 65535 and malformed rows are rejected so the processing will continue with the next row.

```
164 fi
165 if ! [[ "${port}" =~ ^[0-9]+$ ]] || ((port < 1 || port > 65535)); then
166     log_error "Rejected inventory line ${line_number}: invalid port '${port}'"
167     rejected=$((rejected + 1))
168     continue
169 fi
```

Figure 7: Invalid port validation

FR-S02: Command line interface

The script supports the following options;

- -i inventory path
- -o output directory
- -t timeout
- -r report only mode
- -h help

```

60 while getopts ":i:o:t:rh" opt; do
61     case "${opt}" in
62         i) INVENTORY_PATH="${OPTARG}" ;;
63         o) OUTPUT_DIR="${OPTARG}" ;;
64         t) PROBE_TIMEOUT="${OPTARG}" ;;
65         r) REPORT_ONLY=1 ;;
66         h) usage; exit 0 ;;
67         \?)
68             echo "Error: unknown option -${OPTARG}" >&2
69             usage >&2
70             exit 2

```

Figure 8: commandline arguments

```

87 # --- Validate timeout: must be a positive whole number ---
88 if ! [[ "${PROBE_TIMEOUT}" =~ ^[1-9][0-9]*$ ]]; then
89     echo "Error: timeout must be a positive integer, got '${PROBE_TIMEOUT}'" >&2
90     usage >&2
91     exit 2
92 fi

```

Figure 9: timeout validation set to a positive integer

```

94 # --- Validate / prepare output directory ---
95 if [[ ! -d "${OUTPUT_DIR}" ]]; then
96     if ! mkdir -p "${OUTPUT_DIR}" 2>/dev/null; then
97         echo "Error: output directory '${OUTPUT_DIR}' could not be created" >&2
98         exit 2
99     fi
100 fi
101 if [[ ! -w "${OUTPUT_DIR}" ]]; then
102     echo "Error: output directory '${OUTPUT_DIR}' is not writable" >&2
103     exit 2
104 fi

```

Figure 10: output directory created and checked for write permissions

FR-S03: Reachability probing

Using `ping_host()` function it sends one ICMP ping only and waits only for the configured timeout.

```

59 if output="$(ping -c 1 -W "${timeout}" "${address}" 2>/dev/null)"; then
60     rtt="$(printf '%s\n' "${output}" | sed -n 's/.*time=\\([^\ ]*\\).*/\\1/p' | head -n 1)"
61     printf 'UP|s\n' "${rtt:-unknown}"
62 else
63     printf 'DOWN|n'
64 fi
65 }

```

Figure 11: `ping_host()` - `helper.sh`

```

176 probe_result="$(ping_host "${address}" "${PROBE_TIMEOUT}")"
177 status="${probe_result%|*}"
178 rtt="${probe_result#*|}"

```

Figure 12: Returned status and RTT

FR-S04: History, logging & helper library

The helper library provides `log_infor`, `log_error`, `trim`, `write_history` and `ping_host`. The logging functions sends messages to the standard output and errors to standard error.

```

18 set -euo pipefail
19
20 # --- Script-relative paths (never rely on the caller's working directory) ---
21 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
22
23 # shellcheck source=lib/helpers.sh
24 source "${SCRIPT_DIR}/lib/helpers.sh"

```

Figure 13: `netwatch.sh` the strict mode is enabled, use a script relative path to source the shared library

```

22 log_error() {s
23     local message="$1"
24     printf "[%s] ERROR: %s\n" "$(date '+%Y-%m-%d %H:%M:%S')" "${message}" >&2
25 }

```

Figure 14: `log_error()` in `helper.sh`

FR-S05: History analysis and reporting

The report is created with a timestamped filename and it has both bash and python fields. The history records are read and written to the final report it calculates total checks, UP checks, DOWN checks, availability percentage, availability by host and the most frequent failing services.

```

200 report_path="${OUTPUT_DIR}/netwatch_$(date '+%Y%m%dT%H%M%S').csv"
201 TMP_REPORT="${report_path}.tmp"
202 printf 'timestamp,hostname,ip_address,service,port,status,rtt,python_state,python_response_time\n' > "${TMP_REPORT}"
203 if [[ -s "${HISTORY_FILE}" ]]; then
204     while IFS=, read -r timestamp hostname address service port status rtt; do
205         python_state=""
206         python_time=""
207         if [[ -n "${python_report:-}" ]]; then
208             python_row="$(awk -F, ' -v host="${hostname}" -v wanted_port="${port}" 'NR > 1 && $2 == host && $5 == wanted_port {print; exit}' "${python_report}")"
209             if [[ -n "${python_row}" ]]; then
210                 IFS=, read -r _ _ _ _ _ python_state python_time _ <<< "${python_row}"
211             fi
212         fi
213         printf '%s,%s,%s,%s,%s,%s,%s,%s,%s\n' "${timestamp}" "${hostname}" "${address}" "${service}" "${port}" "${status}" "${rtt}" "${python_state}" "${python_time}" >>
214         done < "${HISTORY_FILE}"
215     fi
216 mv "${TMP_REPORT}" "${report_path}"

```

5. PYTHON IMPLEMENTATION

FR-P01: Commandline configuration

The code defines the shared exit codes and only has three accepted modes which are sequential, concurrent and benchmark. The arguments are parsed manually from the list and invalid arguments outputs a error and usage message.

```
106
107     elif arg in ("-t", "--timeout"):
108         value = _next_value(argv, i, arg)
109         try:
110             timeout = float(value)
111         except ValueError:
112             raise ValueError(f"--timeout must be numeric, got '{value}'")
113         if timeout <= 0:
114             raise ValueError("--timeout must be greater than 0")
115         config["timeout"] = timeout
116         i += 1
117
118     elif arg in ("-w", "--workers"):
119         value = _next_value(argv, i, arg)
120         try:
121             workers = int(value)
122         except ValueError:
123             raise ValueError(f"--workers must be an integer, got '{value}'")
124         if workers <= 0:
125             raise ValueError("--workers must be greater than 0")
126         config["workers"] = workers
127         i += 1
```

Figure 15: Getting arguments manually parsed from the list.

```
14  EXIT_SUCCESS = 0
15  EXIT_WARNING = 1
16  EXIT_USAGE_ERROR = 2
17  EXIT_FATAL_ERROR = 3
18  EXIT_INTERRUPTED = 130
```

Figure 16: exit code - netwatch.py

```
28  VALID_MODES = ("sequential", "concurrent", "benchmark")
```

Figure 17: valid modes - netwatch.py

```
585  if __name__ == "__main__":
586      try:
587          sys.exit(main(sys.argv[1:]))
588      except KeyboardInterrupt:
589          print("\nInterrupted - shutting down cleanly.", file=sys.stderr)
590          sys.exit(EXIT_INTERRUPTED)
```

Figure 18: error for Invalid arguments & usage message

FR-P02: Inventory loading and validation

The common inventory is read using csv.reader and the expected header , address and port is checked and validated.

```
184     expected_header = ["hostname", "ip_address", "service", "port", "critical"]
185     if [field.strip() for field in rows[0]] != expected_header:
186         raise ValueError("inventory header must be hostname,ip_address,service,port,critical")
187
```

Figure 19: csv.reader to read the common inventory - netwatch.py

```
152 # FR-P02 - Inventory loading / validation
153
154 def validate_ipv4(address):
155     """Return True if address is a syntactically valid IPv4 address."""
156     match = IPV4_PATTERN.match(address.strip())
157     if not match:
158         return False
159     return all(0 <= int(octet) <= 255 for octet in match.groups())
160
161
162 def validate_port(port_str):
163     port_str = port_str.strip()
164     if not port_str.isdigit():
165         raise ValueError(f"port '{port_str}' is not numeric")
166     port = int(port_str)
167     if not (1 <= port <= 65535):
168         raise ValueError(f"port {port} is out of range (1-65535)")
169     return port
```

Figure 20: validates the ip headers , ports and ipv4 address

```
35 # Data structures
36 @dataclass
37 class Target:
38     hostname: str
39     ip_address: str
40     service: str
41     port: int
42     critical: bool
```

Figure 21: dataclasses

FR-P03: Single TCP service check

Tests the application level availability instead of just only checking the host reachability.

```
260 sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
261 try:
262     sock.settimeout(timeout)
263     # connect_ex returns an errno (0 = success) instead of raising,
264     # which is what FR-P03 asks for: failure as a value.
265     result = sock.connect_ex((target.ip_address, target.port))
266     if result == 0:
267         state = "OPEN"
268     elif result in (111, 10061):
269         state = "CLOSED"
270         error_detail = "connection refused"
271     elif result in (110, 10035, 10060):
272         state = "TIMEOUT"
273         error_detail = "connection timed out"
274     else:
275         state = "ERROR"
276         error_detail = os.strerror(result)
277 except socket.timeout:
278     state = "TIMEOUT"
279     error_detail = "connection timed out"
280 except socket.gaierror as exc:
281     state = "ERROR"
282     error_detail = f"address resolution failed: {exc}"
283 except OSError as exc:
284     # Covers e.g. "Network is unreachable" and similar OS-level errors.
285     if getattr(exc, "errno", None) in (111, 10061):
286         state = "CLOSED"
287         error_detail = "connection refused"
288     else:
289         state = "ERROR"
290         error_detail = str(exc)
291 finally:
292     sock.close()
```

Figure 22: TCP socket and set timeouts and returns error value

```
296     return {
297         "timestamp": timestamp,
298         "hostname": target.hostname,
299         "ip_address": target.ip_address,
300         "service": target.service,
301         "port": target.port,
302         "critical": "yes" if target.critical else "no",
303         "state": state,
304         "response_time": response_time,
305         "error_detail": error_detail,
306     }
```

Figure 23: Contains the information needed by the reports -netwatch.py

FR-P04: Sequential and concurrent scanning

The sequential mode checks each target in order and the concurrent mode uses ThreadPoolExecutor in netwatch.py.

```
311 def run_sequential(targets, timeout):
312     start = time.monotonic()
313     results = [check_service(target, timeout) for target in targets]
314     duration = round(time.monotonic() - start, 4)
315     return _sort_results(results), duration
316
317
318 def run_concurrent(targets, timeout, max_workers):
319     start = time.monotonic()
320     results = []
321     workers = min(max_workers, len(targets)) or 1
322
323     with ThreadPoolExecutor(max_workers=workers) as executor:
324         futures = {
325             executor.submit(check_service, target, timeout): target
326             for target in targets
327         }
328         for future in as_completed(futures):
329             target = futures[future]
```

Figure 24: sequential and concurrent scan modes

```
352 def _sort_results(results):
353     return sorted(results, key=lambda r: (r["hostname"], r["port"]))
```

Figure 25: Results get sorted

FR-P05: Persistence and change detection

The previous results are loaded from JSON and the current services are compared using address and port pairs.

The current results are sent to data/results.json.

```
363     with open(path, "r", encoding="utf-8") as handle:
364         data = json.load(handle)
365         if not isinstance(data, list):
366             raise ValueError("results.json did not contain a list")
367         return data, "previous results loaded"
368     except (json.JSONDecodeError, ValueError) as exc:
369         log_error(f"previous results file was corrupt ({exc}) - treating as first run")
370         return None, "previous results file was corrupt - treated as first run"
371
372
373 def detect_changes(current_results, previous_results):
374     current = {(r["ip_address"], r["port"]): r["state"] for r in current_results}
```

Figure 26: compare the current and previous services

```

413 FR-P05: write reports/py_results_{timestamp}.csv for Part A to merge.
414
415 DATA CONTRACT (confirm with Members 1 & 2 for INT-03):
416   Field order : timestamp,hostname,ip_address,service,port,critical,state,response_time,error_detail
417   Delimiter   : comma
418   Quoting     : csv.QUOTE_MINIMAL (fields are only quoted if they
419                 contain a comma, quote character or newline)
420   Timestamp   : ISO 8601, seconds precision, e.g. 2026-09-25T14:30:00
421   """
422   os.makedirs(reports_dir, exist_ok=True)
423   run_timestamp = datetime.now().strftime("%Y%m%dT%H%M%S")
424   report_path = os.path.join(reports_dir, f"py_results_{run_timestamp}.csv")

```

Figure 27: data contract used

FR-P06: Summary and exit code

Using counts the availability is calculated and the summary ends with a clear status line.

```

440 # FR-P06 - summary and exit code
441
442 def compute_summary(results):
443     counts = {"OPEN": 0, "CLOSED": 0, "TIMEOUT": 0, "ERROR": 0}

```

Figure 28: count states

```

495 def determine_exit_code(results):
496     """FR-P06: map the outcome to the shared exit-code scheme."""
497     if not results:
498         return EXIT_WARNING
499     if any(r["state"] in ("CLOSED", "TIMEOUT", "ERROR") for r in results):
500         return EXIT_WARNING
501     return EXIT_SUCCESS

```

Figure 29: Exit code

```

489 exit_code = determine_exit_code(results)
490 status_word = "OK" if exit_code == EXIT_SUCCESS else "ISSUES DETECTED"
491 print(f"STATUS: {status_word} (exit code {exit_code})")
492 return exit_code

```

Figure 30: summary end including the status

FR-P07: Published status service

The status_server.py reads the newest results.json and the results are sent over HTTP using ThreadingHTTPServer. The default service is 127.0.0.1:8888 and the server makes a readable HTML table from the stored results.

```

32 for item in data:
33     page.append("<tr>" + "".join(
34         f"<td>{html.escape(str(item.get(field, '')))}</td>"
35         for field in ("hostname", "ip_address", "service", "port", "state")
36     ) + "</tr>")
37 page.append("</table></body></html>")
38 self.wfile.write("".join(page).encode("utf-8"))
39 print(f"Served status request to {self.client_address[0]}", flush=True)

```

Figure 31: status_server.py status service

FR-P08: Socket client

The `status_client.py` creates a TCP socket and connects to the status server and it sends an HTTP request. IT gets a complete response in a loop where the bytes are decoded into readable text and finally the socket is closed.

```
5  def fetch_status(host, port):
6      client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7      client.settimeout(3.0)
8
9      try:
10         print(f"Connecting to NetWatch status server at {host}:{port}...")
11         client.connect((host, port))
12
13         # Send HTTP GET request
14         request = "GET / HTTP/1.1\r\nHost: localhost\r\nConnection: close\r\n\r\n"
15         client.sendall(request.encode("utf-8"))
16
17         # Receive binary response
18         response = b""
19         while True:
20             data = client.recv(1024)
21             if not data:
22                 break
23             response += data
24
25         print("\n--- Received Server Reply ---")
26         # Convert bytes to text using .decode()
27         print(response.decode("utf-8"))
28         print("-----\n")
```

Figure 32: client socket - `status_client.py`

6. INTEGRATION

6.1 Integration and pipeline

The main point of integration is the calling of `netwatch.py` by the shell script. While running the shell normally after performing the ping scans bash launches the python script with the same inventory file and timeout. The latter is asked to perform scans concurrently, saving its JSON output and report in the common project directories. Thus bash and python run separately and communicate with each other by means of files not shared memory.

The python CSV report serves as an interface between the two implementations. This report has a definite set of columns such as hostname, IP address, port, service state and response time. After python finishes the shell script locates the most recently modified `py_result_*.csv` file and matches its rows against the bash history using the hostname and port. After executing the shell script finds the newest report created by python and maps its rows according to the shell history based on hostname and port columns. After that the corresponding python state and response time are added to the consolidated report. Therefore the consolidated report may display both ping and TCP service results for the same target.

If the script runs in report only mode then the python scan is not executed and the report is built solely from the shell history. These elements use the same inventory but still have different result formats for themselves the bash script appends the history of pings to a CSV file whereas the python script creates a new JSON file with the current state of the services and also a new CSV file for the purpose of integration.

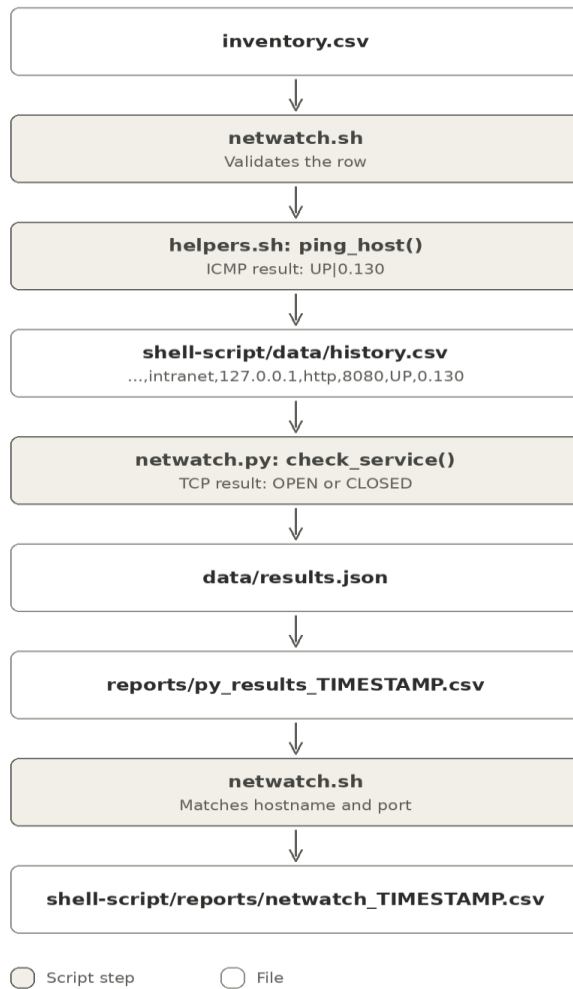


Figure 33: pipeline flow

6.2 Technology Justification

Part A: Bash – Host level reachability

Capability	Why bash
Reading CSV-like text row by row	Shell loops read line-delimited text natively, with no parsing library needed for a simple inventory file.
Native regex with <code>[[... =~ ...]]</code>	Validates fields such as hostnames and IP addresses directly in the shell without needing an external tool.
Invoking ping	ICMP checks are a one line call to a standard OS utility and the output can be easily captured
awk , sed , sort , printf	Having Standard Unix text tools handle parsing, cleaning, ordering, and formatting the report with minimal code.
Scheduling through cron	Bash scripts run under cron with no extra runtime or service manager.
Simple logs and delimited history files	Appending to plain-text CSV history is trivial with redirection (<code>>></code>).

Part B: Python – server level availability

Capability	Why Python
TCP socket programming	The socket module opens connections to a specific port which is what a per service check needs.
Explicit timeout and exception handling	<code>settimeout()</code> and <code>try/except</code> separate open, closed, refused and timed-out cases cleanly.
Structured target and result records	Classes or dataclasses give each check a typed, readable structure instead of loose strings.
Concurrent checks with ThreadPoolExecutor	Many ports can be checked in parallel so slow or timing out targets don't block the whole run.
JSON persistence	The json module saves and reloads results with their structure intact.
Set based change detection	Set operations compare the previous and current results to find newly opened or closed services.
Serving results through an HTTP server	The standard library can expose results over HTTP without a web framework.
Implementing a socket client	The same socket module supports the client side so one language covers both roles.

7. TESTING & RESULTS

7.1 Test Plan

Author	Test ID	Category	Test case	File / Function tested	Test method	Expected result
Member 1	TC-A-13	Negative	Reject malformed Ipv4 address and continue processing	Ipv4 validation	Run netwatch.sh using an inventory containing 999.1.1.1 followed by a valid 127.0.0.1 entry	Invalid IPv4 row is rejected with a clear reason, while valid rows continue to be processed
Member 1	TC-A-14	Negative	Reject invalid port numbers	Port validation	Run the script using ports 0 and 65536	Invalid IPv4 row is rejected with a clear reason, while valid rows continue to be processed
Member 1	TC-A-15	Functional	Probe a reachable host and extract RTT	lib/helpers.sh – ping_host()	Execute source lib/helpers.sh; ping_host 127.0.0.1 2	Function returns UP <RTT>
Member 1	TC-A-16	Negative / Failure	Handle a failed ping command under strict mode	Lib/helper.sh – ping_host()	Temporarily replace ping with a command returning exit code 1, then execute under set -euo pipefail	Function returns DOWN and the shell does not terminate unexpectedly
Member 2	TC-A-04	Functional	Verify the script passes static analysis with no syntax errors and no ShellCheck findings	netwatch.sh, lib/helpers.sh	Run bash -n netwatch.sh and shellcheck -x netwatch.sh lib/helpers.sh	Full usage message is displayed and exit code is 0
Member 2	TC-A-05	Functional	Verify -h prints usage information and exits cleanly	netwatch.sh, lib/helpers.sh	Run ./netwatch.sh -h; echo "exit: \$?"	Full usage message is displayed and exit code is 0
Member 2	TC-A-10	Functional	Verify Bash invokes Python and	netwatch.sh +	Run ./netwatch.sh -i config/inventory.csv -o reports -t 2 with a	Bash performs reachability checks, invokes

			displays a combined per-service results table	netwatch.h.py	local HTTP server running on port 8080	Python, and displays the combined HOSTNAME / SERVICE / PORT / STATE / RESP(s) table
Member 2	TC-A-11	Functional	Verify Python results are correctly joined into the merged CSV report	netwatch.h.sh, Python results CSV	Run an orchestrated scan and inspect the newest reports/netwatch_*.csv	Merged CSV contains Bash and Python fields correctly aligned; response time is populated for open services
Member 2	TC-A-12	Functional	Verify report-only mode regenerates a report without rescanning or appending history	netwatch.h.sh -r mode	Record data/history.csv line count, run ./netwatch.sh -r -o report-only, then compare the line count	History remains unchanged, a new report is generated, and exit code is 0
Member 3	TC-B-01	Functional	Verify successful TCP connections are identified as OPEN	Python NetWatch TCP service checking (FR-P03)	Start temporary TCP listeners on 127.0.0.1 ports 8080, 8081, and 9000, then scan the three-target inventory sequentially with a 0.5s timeout	All three targets are reported as OPEN and the program exits successfully
Member 3	TC-B-02	Functional	Verify consistent results between sequential and concurrent modes and measure durations	Python NetWatch sequential/concurrent scanning (FR-P04)	Scan six targets in both modes using a 0.2s timeout and four workers. Three ports have listeners and three do not. Run benchmark mode	Both modes produce matching target states: 3 OPEN and 3 CLOSED; benchmark displays both durations
Member 3	TC-B-03	Negative	Verify an out-of-range port is rejected while valid rows continue	Python NetWatch inventory validation (FR-P02)	Use an inventory containing valid targets before and after a row containing port 70000	Invalid row is rejected with its line number and reason; valid rows continue processing and are checked successfully
Member 3	TC-B-04	Failure	Verify a service stopping	Python NetWatch change	Scan 127.0.0.1:8080 while its listener is active, stop the	First run reports OPEN; second run reports CLOSED

			between runs is detected	detection (FR-P05)	listener, then scan again using the same inventory and saved-results location	and identifies the endpoint under Newly closed
Member 3	TC-B-05	Boundary	Verify inventory handling with zero targets	python/netwatch.py inventory handling (FR-P02)	Run a sequential scan using an inventory containing only the valid CSV header and no target rows. Supply the inventory through Bash process substitution.	Reports 0 accepted and 0 rejected rows, displays “no valid targets to check”, performs no target checks and exits with code 1. Reports 0 accepted and 0 rejected rows, displays “no valid targets to check”, performs no target checks and exits with code 1.
Member 4	TC-B-06	Boundary	Verify the minimum and maximum valid port values are accepted	Python/netwatch.py port validation and TCP service checking (FR-P02, FR-P03)	Run two separate sequential scans against 127.0.0.1, each with a single-target inventory: one using port 1 and the other port 65535. Use a 0.5-second timeout.	Each run accepts one row without a port-validation error and produces a result for the target. OPEN, CLOSED or TIMEOUT is acceptable because this test checks valid boundary-port acceptance, not service availability.
Member 4	TC-B-07	Functional	Verify status_server.py publishes current status over HTTP	status_server.py (FR-P07)	Run status_server.py and execute curl http://127.0.0.1:8888	HTTP 200 response containing an HTML table displaying target host states
Member 4	TC-B-08	Functional	Retrieve published status using the custom socket client	status_client.py (FR-P08)	Run status_client.py while the status server is active	Client connects, receives the byte buffer, decodes and displays the status summary, then closes the socket
Member 4	TC-B-09	Negative	Handle client execution	status_client.py	Run status_client.py when no server is	ConnectionRefusedError is caught

			when the status server is offline		listening on port 8888	cleanly, a user message is displayed, and the program exits with a non-zero status
Member 4	TC-B-10	Failure	Recover server startup when the target port is already in use	status_server.py	Launch two instances of status_server.py using port 8888	Second instance catches OSError: Address in use and exits cleanly without a traceback

7.2 Results table

Test ID	Test command / Data	Actual Result	Status
TC-A-13	<code>./netwatch.sh -i /tmp/tc-invalid-ip.csv -o /tmp/tc1-reports -t 1 2>&1 tee /tmp/tc1-output.txt</code>	Inventory line containing 999.1.1.1 was rejected as an invalid IPv4 address. The output reports 2 accepted and 1 rejected. The valid 127.0.0.1 entries continued to be processed.	Pass
TC-A-14	<code>./netwatch.sh -i /tmp/tc-invalid-port.csv -o /tmp/tc2-reports -t 1 2>&1 tee /tmp/tc2-output.txt</code>	Port 0 and port 65536 were both rejected as invalid/out of range. The output reports 1 accepted and 2 rejected. Processing continued with the valid row.	Pass
TC-A-15	<code>source lib/helpers.sh; ping_host 127.0.0.1 2</code>	The function returned UP 0.130 in one execution and UP 0.112 in the repeated execution, demonstrating that the hTC-A-04ost was reachable and an RTT value was successfully extracted.	Pass
TC-A-16	Fake ping executable returning exit code 1, executed using <code>set -euo pipefail</code>	The function returned Result: DOWN and the shell remained active, demonstrating safe handling of the failed ping command.	Pass
TC-A-04	<code>bash -n netwatch.sh; shellcheck -x netwatch.sh lib/helpers.sh</code>	<code>bash -n</code> produced no output, indicating no syntax errors. The initial ShellCheck run identified SC2012 relating to an ls-based file listing. This was replaced with a find-based equivalent and the test was	Pass

		rerun from the correct directory, producing zero finding	
TC-A-05	<code>./netwatch.sh -h; echo "exit: \$?"</code>	The complete usage text was displayed, including the -i, -o, -t, -r and -h options and their defaults. The command returned exit: 0.	Pass
TC-A-10	<code>./netwatch.sh -i config/inventory.csv -o reports -t 2</code> with a local HTTP server on port 8080	Bash successfully performed host reachability checks and invoked the Python component. The combined results table was displayed with HOSTNAME, SERVICE, PORT, STATE and RESP(s). The live service on port 8080 was shown as OPEN, while the remaining services were CLOSED.	Pass
TC-A-11	Inspect newest reports/netwatch_*.csv after an orchestrated run	The merged CSV contained timestamp, hostname, ip_address, service, port, status, rtt, python_state and python_response_time. RTT/response-time fields were blank for DOWN services and populated for UP services, matching the Python results CSV	Pass
TC-A-12	Record history line count, then run <code>./netwatch.sh -r -o report-only</code>	The history file remained unchanged: before = 60 lines, after = 60 lines. A new report file was created in the report-only directory and the command returned exit: 0.	Pass
TC-B-01	Sequential scan using a temporary inventory containing 127.0.0.1:8080, 8081 and 9000; timeout 0.5s.	All three targets were reported as OPEN. Exit code was 0	Pass
TC-B-02	Sequential and concurrent scans of shell-script/config/inventory.csv; timeout 0.2s, with 4 workers for concurrent mode; benchmark mode used to compare durations.	Both modes produced matching results: 3 OPEN and 3 CLOSED. Sequential duration was 0.0006 s, concurrent duration 0.0020 s, with reported speed-up of 0.30×. Concurrent mode was slower in this run.	Pass
TC-B-03	Sequential scan using an inventory containing valid	Inventory line 3 was rejected because port 70000 was	Pass

	ports 8080, 8081 and invalid port 70000; timeout 0.5s.	outside the valid 1–65535 range. 2 accepted, 1 rejected. Both valid targets were reported as OPEN and exit code was 0.	
TC-B-04	Two sequential scans using the same inventory; port 8080 running during the first scan and stopped before the second scan; timeout 0.5s	First run reported OPEN with exit code 0. Second run reported CLOSED with exit code 1. The change summary listed ('127.0.0.1', 8080) u	Pass
TC-B-05	Sequential scan using a CSV inventory containing only the valid header, with no target rows.	Reported 0 accepted, 0 rejected and “no valid targets to check”. No targets were checked; change lists were empty. Displayed exit code 1.	Pass
TC-B-06	Two separate sequential scans of 127.0.0.1, using single-target inventories with ports 1 and 65535, and a 0.5-second timeout.	Each run reported 1 accepted, 0 rejected. Both boundary ports were checked and returned CLOSED, with a response time of 0.0001 s each. Both runs displayed exit code 1.	Pass
TC-B-07	Start status_server.py and run curl http://127.0.0.1:8888	The server returned an HTTP 200 response containing an HTML table showing the current target host states.	Pass
TC-B-08	Run status_client.py while status_server.py is active	The client successfully connected, received the byte buffer, decoded the response, displayed the status summary and closed the socket cleanly.	Pass
TC-B-09	Stop the Python server and run status_client.py with no server listening on port 8888	ConnectionRefusedError was caught cleanly, an appropriate user message was displayed and the program exited with a non-zero status.	Pass
TC-B-10	Launch two instances of status_server.py on port 8888	The second instance caught OSError: Address in use and exited cleanly without producing a traceback	Pass

7.3 Performance testing

Performance measurements were carried out in Ubuntu WSL using benchmark mode. Each benchmark compared sequential and concurrent scanning using the same inventory with each workload repeated three times. The standard inventory contained six targets. A larger test

used 100 consecutive ports on 127.0.0.1, covering ports 30000–30099 with 10 worker threads and a 0.5 second connection timeout. The output files were saved to temporary locations. Speed-up is calculated by dividing the mean sequential time by the mean concurrent time. A value below 1 indicates that concurrent mode was slower. Since the measured times are very short, small differences in system scheduling can have a noticeable impact on the results.

Workload	Sequential runs (s)	Concurrent runs (s)	Mean sequential (s)	Mean concurrent (s)	Speed up *
Submitted inventory, 6 targets	0.0005, 0.0005, 0.0005	0.0062, 0.0058, 0.0046	0.0005	0.0055	0.09
Localhost range, 100 targets	0.0056, 0.0055, 0.0084	0.0622, 0.0669, 0.0569	0.0065	0.0620	0.10*

Result:

Concurrent mode was slower than sequential mode for the two localhost workloads tested. However these results only apply to the conditions used in the experiment and should not be interpreted as evidence that concurrent scanning is always slower. Workloads involving slower connections or longer waiting times may benefit more from concurrent execution.

8. ERROR HANDLING

The selected tests checked how the system handles common errors and unexpected situations. The main issues were related to code quality, command-line usage, Bash–Python integration, report data, and report-only mode.

8.1 Error-Handling Condition Matrix

Error / Condition	Test Case	Result
ShellCheck warning found in the script	TC-A-04	Pass – The SC2012 warning was fixed and the final check produced no findings.
Help option requested	TC-A-05	Pass – Usage information was displayed and the program exited normally.
Closed services returned by Python	TC-A-10	Pass – OPEN and CLOSED services were correctly shown in the combined table.
Python results joined with Bash results	TC-A-11	Pass – CSV fields were correctly matched and response times were populated where applicable.
Report generated without rescanning	TC-A-12	Pass – The report was regenerated without changing the existing history.

8.2 Error Handling Discussion

In TC-A-04, ShellCheck initially identified an SC2012 warning caused by the use of an `ls`-based file listing. This was replaced with a `find`-based approach, and the final check produced no warnings.

TC-A-05 confirmed that the `-h` option works correctly. The program displayed the full usage information and exited with code 0 without starting a scan.

In TC-A-10, the Bash script successfully called the Python component and displayed the combined results. The live service on port 8080 was shown as OPEN, while the unavailable services were shown as CLOSED.

TC-A-11 confirmed that the Bash and Python results were correctly combined in the CSV report, with response times shown for available services and left blank when the service was down.

Finally, TC-A-12 verified that report-only mode could generate a new report without performing another scan or adding new history records. The history remained at 60 lines before and after the test.

Overall, all five selected test cases passed, with the issue found during TC-A-04 corrected before the final testing.

9. SECURITY CONSIDERATIONS

Authorization and scan scope: The inventory supplied with NetWatch contains six targets, all using the loopback address 127.0.0.1. The README also states that the tool should be used only against localhost or a lab range authorized in writing. However, this boundary is not enforced by either program. They validate that an address is syntactically valid IPv4, but do not check whether it belongs to an approved range. An operator could therefore edit the inventory to include another valid address. Before deployment, NetWatch should check every target against an explicit allowlist of authorized addresses or CIDR ranges and reject out-of-scope entries before making any network connection.

Input validation and shell safety: The inventory is intended to be edited by non-programmers, so malformed or hostile values must not disrupt a run. Python uses `csv.reader`, validates the address and port, and restricts hostname and service values to a limited character set. Bash checks the address octets, port range, required fields and critical flag. The validation rules are not identical: Bash accepts any non-empty hostname or service value and requires lowercase yes or no, while Python applies character restrictions and accepts those flags case-insensitively. Bash passes expanded variables as quoted arguments, so shell metacharacters in a field are not reparsed as commands. Quoting is important because an unquoted variable can undergo word splitting and wildcard expansion; for a delete operation, `--` also protects a path that begins with a dash. The temporary report path is quoted in the current cleanup command. However, Bash writes field values directly to CSV, so inconsistent input validation can still produce malformed report data.

Status-service exposure: The status server listens on 127.0.0.1:8888 by default. This limits access to processes on the same machine, which is suitable for the coursework setup, but it does not authenticate local users or processes. The `--host` option can bind the service to another interface. The server uses HTTP without authentication or TLS, so a wider binding could expose hostnames, IP addresses, ports and service states to other devices. Values displayed in the HTML page are escaped, which helps prevent them being interpreted as markup; it does not prevent disclosure of the network information itself.

File permissions and logs: NetWatch does not explicitly set restrictive permissions for its logs, history, reports or JSON results. Their confidentiality therefore depends on the account that runs the tool, the directory permissions and the system umask. The files contain network and service details that could help someone map the environment or identify unavailable services. Write access to the scripts, inventory or scheduled-run configuration could also let someone change scan targets or alter code that runs on a schedule. Access should be limited to the administrator and authorized IT users, with the tool run under an account that does not have unnecessary privileges.

10. CRITICAL EVALUATION & CONCLUSION

NetWatch addresses the main problem in the scenario by checking both host reachability and TCP service availability. The submitted results show that malformed IP and port entries were rejected while valid entries continued to be processed. The failed-ping test also shows that an expected non-zero ping result is recorded as DOWN without terminating the Bash script under `set -euo pipefail`. The report records a clean ShellCheck rerun after an issue was fixed, a successful orchestrated run, Python results appearing in the merged report, and report-only mode leaving the history file at 60 lines before and after the run. These results support the claim that several important Bash and integration behaviors work under the tested conditions.

The use of two languages is a practical design choice for this small tool. Bash provides a direct way to run ICMP checks and schedule the script with cron, while Python's socket library and ThreadPoolExecutor support TCP checks and concurrent scanning without additional packages. The expected failure from ping is handled inside a conditional in `ping_host()`, allowing strict mode to remain enabled while the scan continues. In concurrent mode, each worker reads a target and creates its own socket; the main thread gathers and sorts the results. This avoids workers writing to a shared results list at the same time. The trade-off is that the two components have separate inventory validation logic and exchange results through files, which makes consistent validation and accurate result matching harder to maintain.

Several limitations remain. The scan boundary is stated in the README but is not enforced in code, and the Bash and Python validation rules differ despite the report's claim that they are the same. The Bash availability summary uses the entire accumulated history, with no time window or retention policy, so old failures continue to affect the reported percentage. During integration, the newest Python CSV is used when Bash builds the report from its history; as a result, older Bash records can be shown alongside a later Python service state. The status server reads `results.json` while the scanner writes it directly, so a request during a write could encounter incomplete JSON.

The concurrent design is appropriate for the six-target inventory, but the submitted evidence does not establish a maximum viable workload. The worker count is capped by the configured limit, which defaults to 10, but the program loads the inventory and creates a future for each target. For scans dominated by timeouts, sequential duration grows roughly with the number of targets multiplied by the timeout; concurrency reduces waiting, with additional costs from queueing and result handling. The report does not include the required measurements for two workload sizes repeated three times, so a quantitative speed-up cannot yet be claimed. The actual-result entries for the Python test cases are also incomplete.

The first improvement should be an enforced scan allowlist, because it directly reduces the risk of probing an unauthorized network. A second improvement should align validation between Bash and Python and associate both components' results with the same run, reducing inconsistent records and misleading historical joins. This would require a moderate change to the integration. Finally, adding history retention or time-windowed availability calculations

would keep long-term reports useful as data grows. These changes would make NetWatch safer and more dependable before it is used beyond the small coursework test estate.

Overall, the project demonstrates a working monitoring prototype for the supplied local test setup, with evidence for key Bash and integration behaviors. The available report does not yet verify every requirement: the scan boundary is not enforced, the validation rules differ, Python test results are incomplete, and the performance target has not been demonstrated with the required measurements. NetWatch should therefore be treated as suitable for the tested coursework environment rather than as a fully validated production monitoring system.

REFERENCES

GNU Project (2025) *Bash Reference Manual*. Available at:
<https://www.gnu.org/software/bash/manual/bash.html> (Accessed: 28 September 2026).

Python Software Foundation (2024a) *http.server* — *HTTP servers*. Available at:
<https://docs.python.org/3/library/http.server.html> (Accessed: 27 September 2026).

Python Software Foundation (2024b) *socket* — *Low-level networking interface*. Available at:
<https://docs.python.org/3/library/socket.html> (Accessed: 27 September 2026).

Python Software Foundation (n.d.a) *concurrent.futures* — *Launching parallel tasks*.
Available at: <https://docs.python.org/3/library/concurrent.futures.html> (Accessed: 27 September 2026).

Python Software Foundation (n.d.b) *json* — *JSON encoder and decoder*. Available at:
<https://docs.python.org/3/library/json.html> (Accessed: 27 September 2026).

Python Software Foundation (n.d.c) *Built-in types: Set types* — *set, frozenset*. Available at:
<https://docs.python.org/3/library/stdtypes.html#set-types-set> (Accessed: 27 September 2026).

ShellCheck (n.d.) *SC2012: Use find instead of ls for parsing*. Available at:
<https://www.shellcheck.net/wiki/SC2012> (Accessed: 27 September 2026).

APPENDIX A – Code Listing

<https://github.com/kithmini-jayasinghe/NPD-CW-Group-01>

APPENDIX B

Work Allocation table

Member	Area	Main Contribution	Share %
Member 1- S M J Navarathne	Part A Lead – Bash Core Engine & Library	Implemented FR-S01 Inventory Regex Validation , including line-by-line CSV processing, header handling, IPv4 and port validation, row-level error reporting, and continuation after invalid rows. Implemented FR-S03 Reachability Probing , including bounded ping operations and RTT extraction. Developed reusable functions in lib/helpers.sh, handled the set -euo pipefail and failed-probe interaction, and ensured ShellCheck compliance. Conducted four structured test cases covering validation and reachability.	25%
Member 2 – K A Baylon	Part A & Integration Lead – Bash CLI, Reporting & Pipeline	Implemented FR-S02 CLI Interface using getopts, including inventory, output directory, timeout and report-only options. Implemented FR-S04 Logging & History , including logs/netwatch.log and data/history.csv. Implemented FR-S05 and INT-01–INT-03 , including Python orchestration, exit-code handling, integration of Part B results, and consolidated reporting. Configured the hourly crontab job and documented four structured test cases covering CLI, reporting and integration.	25%
Member 3 – J A K C Jayasinghe	Part B Lead – Python Core Engine & Concurrency	Implemented FR-P01 and FR-P02 , including guarded Python entry points, command-line argument processing, inventory loading and validation. Implemented FR-P03 Socket Checking Engine , including TCP socket connections, timeouts, refused connections, address-resolution errors and socket cleanup. Implemented FR-P04–FR-P06 , including sequential and concurrent scanning, timing measurements, result persistence and set-based change detection. Conducted performance benchmarks and four structured test cases covering socket checking, concurrency and set operations.	25%
Member 4 – A T D Silva	Part B Lead – Python Core Engine & Concurrency	Implemented FR-P01 and FR-P02 , including guarded Python entry points, command-line argument processing, inventory loading and validation. Implemented FR-P03 Socket Checking Engine , including TCP socket connections, timeouts, refused connections, address-resolution errors and socket cleanup.	25%

		Implemented FR-P04–FR-P06 , including sequential and concurrent scanning, timing measurements, result persistence and set-based change detection. Conducted performance benchmarks and four structured test cases covering socket checking, concurrency and set operations.	
--	--	--	--

Contribution statements

I worked on the Bash core engine and library, including inventory CSV validation, IPv4 and port validation, reachability probing, error handling and reusable helper functions. I also carried out testing, debugging and ShellCheck checks for my implementation. In addition I contributed to the project report by writing the relevant technical sections, preparing diagrams and documenting the implementation and testing results

Name: S M J Nawarathna

Student ID: COHNDNE252F-005

Signature: _____

Date: 9/28/2026

I confirm that I contributed to the development and testing of the Bash-based NetWatch functionality assigned to me, covering the CLI, logging and history, orchestration, reporting, and integration requirements **FR-S02, FR-S04, FR-S05, INT-01, INT-02, and INT-03**.

I developed and conducted tests for CLI handling, logging and history, report-only mode, error handling, orchestration, merged reporting, and Ctrl+C interruption. I also investigated issues such as ShellCheck SC2012 and the testing approach for interruption handling.

AI tools were used as supporting resources for development, task organisation, command verification, and testing. I independently ran the tests, reviewed the outputs and evidence, and made the final decisions based on the actual implementation and results.

I confirm that the work and evidence attributed to me accurately represent my contribution to the project.

Name: K A Baylon

Student ID: COHNDNE252F-004

Signature: _____

Date: 9/28/2026

I developed the Python NetWatch component covering FR-P01–FR-P06, including command-line options, inventory validation and TCP service checks. I implemented sequential and concurrent scanning, JSON and CSV output, set-based change detection, and result summaries. I also helped connect the Bash and Python components and combine the work completed by individual members. I tested the Python component and its integration with the overall system using WSL Ubuntu. The most challenging part was understanding how concurrent checks finish in different orders while keeping the results consistent, which I addressed by reviewing how results were collected, sorted and compared using address-and-port pairs.

Name: J.A.K.C. Jayasinghe

Student ID: COHNDNE252F-007

Signature: _____

Date: 9/28/2026

I built the status_server.py HTTP status publishing service (FR-P07) and the custom status_client.py socket client (FR-P08) required for Group 1. I also configured and launched our simulated background HTTP server estate across ports 8080, 8081, and 9000. I created and executed four functional and failure test cases to verify HTTP response formatting, socket data reception, connection timeouts, and port collision handling. Additionally, I wrote Section 9 of the report covering network authorization boundaries and security risks. Dealing with raw binary buffer decoding (.decode()) and ensuring TCP socket connections closed cleanly under connection failures was the trickiest part of the build.

Name: A D T Silva

Student ID: COHNDNE252F-014

Signature: _____

Date: 9/28/2026

APPENDIX C – AI Use Declaration

1. S M J Nawarathna

I used GitHub Copilot in Visual Studio Code to assist with debugging, identifying errors and making corrections during the development of my assigned tasks. I reviewed the suggestions and made all the final decisions regarding the implementation and changes. I also used Claude AI to help generate diagrams for the project report. I independently tested the implemented features in my WSL environment and verified the results against the project requirements. AI tools were used only as supporting tools while the final implementation, testing and decisions were my own.

2. K A Baylon

I used AI assistance during the development, organisation, and testing of my contribution. Claude AI was used to structure requirements, organise tasks, track progress, identify missing checkpoints, and support testing plans. GitHub Copilot in VS Code (Agent) assisted with programming, scripting, testing, and development tasks, while ChatGPT was mainly used to verify and explain terminal, Git, and testing commands.

AI assistance was also used to discuss issues such as ShellCheck SC2012, Ctrl+C interruption testing, and Python exit-code handling. I independently ran the commands and tests in my WSL environment, reviewed the actual outputs and evidence, and made the final decisions regarding implementation and test results. AI was used as a support tool and not as a substitute for my own verification, judgement, or responsibility.

3. J A K C Jayasinghe

AI tools used: OpenAI ChatGPT and GitHub Copilot through Visual Studio Code.

I used ChatGPT to assist with developing, understanding and debugging the Python NetWatch component, including inventory validation, socket checks, concurrency, result persistence and change detection. I also used it to review the code against the coursework requirements and help prepare documentation.

I used GitHub Copilot in Visual Studio Code for implementation assistance and error handling, particularly when connecting the Bash and Python components and combining the parts developed separately by group members.

I reviewed and adapted the generated suggestions, incorporated relevant changes into the project, and participated in testing the combined system using WSL Ubuntu.

4. A T D Silva

Tools Used: Gemini

Purpose: Used as a coding assistant to help set up Python HTTP servers and troubleshoot local Git/GitHub workflows in Ubuntu.

Outcome: Reviewed the suggested Python `http.server` and `socket` code structures, modified the file path handling to match our repository layout (`python/data/results.json`), and manually executed, tested, and verified all scripts locally before committing.